

客户端工程训练营-日报周报提交-吴一鸣

 技术栈：android

0基础

使用codex辅助开发

仓库地址：<https://github.com/whh062413/SingleColumnAdFeed-Android>

录屏



模块拆分方案：

数据模块：

- 广告内容模型（标题、描述、封面、标签、AI 摘要）
- 卡片类型 & 分类定义（大图/小图/视频、推荐/电商/本地）
- Mock 数据 + 封面图
- SQLite 本地持久化（点赞、收藏交互记录）

主界面：

- 单列 主页（Compose LazyColumn）
- 分类 Tab 切换（推荐 / 电商 / 本地）
- 搜索栏
- 下拉刷新
- 骨架动画

详情页面：

- 详情页
- 封面大图展示
- AI 分析卡片（摘要 + 智能标签）
- 点赞交互动画

资源复用：

- 广告卡片工厂组件
- Material 3
- 视频播放器池（复用）
- 状态页组件（Loading / Empty / Error）

交互模块：

- 点赞 / 收藏（SQLite 持久化，重启不丢）
- AI 广告摘要自动生成（云端 DeepSeek）
- 曝光埋点追踪（可见 $\geq 50\%$ 且持续 ≥ 1 秒）
- 点击事件追踪

后端服务：

- OkHttp 网络层
- AI API 服务（接入 DeepSeek）
- AI 搜索，总结
- AI 配置管理中心（API Key / URL / Model）

总体方案

技术栈选型： Android + Kotlin (UI) / Java (数据层) 混合开发，Jetpack Compose 声明式 UI，Material 3

架构设计： 采用 MVVM + Repository 分层架构，UI 层通过 ViewModel 观察状态驱动刷新，Repository 统一聚合 Mock 数据、本地 SQLite 持久化与远程 AI 服务，各层职责解耦、单向依赖

核心功能方案：

- **单列信息流：** Compose LazyColumn 实现卡片瀑布流布局，snapshotFlow 监听滚动位置触发无限加载（距底部 3 项预加载），Material 3 PullToRefreshBox 下拉刷新
- **多类型卡片：** AdCardFactory 工厂模式渲染卡片，Coil 异步图片加载，ExoPlayer 视频播放器池复用
- **分类与搜索：** 三分类 Tab 切换（推荐 / 电商 / 本地），搜索采用 AI 云端拆词优先 + 本地标点分割降级，标题+描述+标签多维模糊匹配
- **AI 智能分析：** 接入 DeepSeek，广告摘要生成 + 智能标签提取，云端优先、本地规则降级，多线程 Executor 隔离（数据/AI/刷新互不阻塞）
- **交互与持久化：** 点赞收藏通过手写 SQLite DAO 持久化，重启不丢失；详情页独立 Activity + Compose，含 AI 分析卡片
- **埋点追踪：** 曝光判定，点击事件记录 feedId 与时间戳
- **网络层：** OkHttp

过程中出现的方案对比

1.开发顺序

一开始看了其他有问题的资料先开发了ui模块，由老师提出问题后重新思考结合ai的思路重新设计了后续的开发流程

2.关于语言

kotlin全部负责关于ui部分的内容，剩下的java

3.线程池

在初次写好后发现刷新和ai分析纵使对冲，用一个另外一个就慢的离谱

旧方案（三独立 Executor） 新方案（viewModelScope 协程）

代码块

```
1  viewModelScope.launch {  
2      refresh()          → 同步生成 Mock → 串行调 AI（协程挂起，不阻塞）  
3      loadMore()         → 同上  
4      search()           → AI 拆词（协程阻塞等待）→ 本地 filter  
5  }
```

4.点赞同步

旧方案（6.11 之前的问题）

FeedScreen 和 DetailScreen 各自维护自己的点赞状态（比如各自存 `mutableStateOf`），点了之后只更新本地，另一个页面看不到变化。回到 FeedScreen 后还是旧的点赞状态。

新方案：SQLite 写入 → Repository 返回新对象 → allItems 赋值 → 所有 `collectAsState()` 观察者自动收到新值

这样就可以一致，重启也不丢

代码块

```
1  FeedViewModel（全局单例）  
2      |  
3      allItems: StateFlow<List<FeedItem>>    ← 唯一真相源  
4      |  
5      |—— FeedScreen.collectAsState()        ← 观察  
6      |—— DetailScreen.collectAsState()      ← 观察  
7  
8      点 like 按钮 → toggleLike(id)  
9      |  
10     allItems.value = newList                ← 不可变新对象赋值  
11     |  
12     |—— FeedScreen 自动重组                ← 两边同时刷新  
13     |—— DetailScreen 自动重组
```

5.视频

旧方案：视频想和图片一样走本地mock存储，但是第一次封包的时候不知道原因电脑就炸了

新方案：url，改了一点逻辑

VideoCacheManager

- |—— getPlayableVideoUri(url)
 - | |—— 直接用远程 URL 播放
- |—— cacheVideo(url)
 - |—— HttpURLConnection 下载 → .tmp 临时文件
 - |—— 校验非空 → rename 为正式缓存文件
 - |—— 文件名 = SHA-256(URL) + ".mp4" （防止非法字符、去重）

FeedVideoPlayerPool (ExoPlayer 复用池)

- |—— Map<String, ExoPlayer> 按 itemId 缓存播放器实例
 - |—— acquire() → 有则复用，无则创建
 - |—— release() → 停止并释放

但这样还是存在问题，即加载视频会有点慢

6.数据存储

旧方案：按照课程设计的SharedPreferences

新方案：SQLite

好处：点赞/收藏结构化

6.4日



学习群中资料客户端开发的课时一、课时二的资料，配置好安卓开发环境，动手创建了 demo-activity，了解了常见布局、UI组件、与数据存储方式

6.5日



学习群中课时三、课时四的资料，学习了其他类似项目的架构

确定项目模块的设计（将学习到的方案和ai的建议进行整合）

UI层(Compose) -> FeedScreen / DetailScreen / 卡片组件

ViewModel层(Java) -> FeedViewModel 唯一状态持有者

Repository层(Java) -> FeedRepository接口 -> DefaultFeedRepository

数据层 -> 本地Mock数据

网络层(Java) -> OkHttpProvider / ai的api接口

语言+UI: Kotlin, Java

架构: MVVM (AndroidViewModel + LiveData + Repository 模式), 单例工厂贯穿各层

列表: LazyColumn + Compose PullToRefreshBox + snapshotFlow 滚动监听触底加载

导航: 双页结构。主页是 MainActivity 直接 Compose; 详情页通过 Intent + Serializable 跳转 DetailActivity

网络请求: OkHttp, Gson 序列化, OkHttpClient 全局单例

数据存储: SharedPreferences

AI 相关: 暂定用deepseek的api

缓存: 按照分页截取子列表返回 (20), 滑动到 (已加载-3) 时加载下一页

6.6日

ui/detail/

DetailScreen.kt 详情页 Compose (ai辅助)

DetailActivity.kt 详情页 Activity 容器, 接收 Intent 传参并挂载 DetailScreen

ui/components/

AdCardFactory.kt 广告卡片: 根据类型渲染 BigImageCover (220dp) 或 SmallImageCover (100dp 缩略图) (ai辅助)

SkeletonLoading.kt 骨架屏占位组件, 首次加载时展示

MainActivity.kt setContent 挂载 FeedScreen, 处理卡片点击跳 DetailActivit

ui/feed/

FeedScreen.kt 主页面 Compose: TopAppBar + CategoryTabs + PullToRefreshBox + LazyColumn 列表 + 底部加载更多 + 骨架屏

6.7日

viewmodel/ (ai辅助)

FeedViewModel.java 首屏加载、分页、下拉刷新、分类切换，LiveData 驱动 UI

FeedUiState.java UI 状态容器 Builder，封装 items 列表

loading/refreshing/loadingMore/hasMore/currentCategory

data/model/

FeedItem.java 广告实体 (ai辅助)

FeedItemType.java 枚举 BIG_IMAGE / SMALL_IMAGE / VIDEO，决定卡片布局类型

FeedCategory.java 决定 Tab 分类

AdInsight.java 用ai分析，含 summary（摘要文本）和 tags（标签列表）（还是本地的）

data/mock/

MockFeedDataSource.kt 按轮转规则用本地文字数据

data/repository/

FeedRepository.java 仓库接口，定义 loadMore / toggleLike / toggleCollect / getItemById

DefaultFeedRepository.java 仓库实现，调 Mock 数据源 + InteractionStore，预留api

6.8日

ui/theme/

Theme.kt 使用Material3定义主题 (ai辅助)

修改ui/detail/

DetailScreen.kt

AdCardFactory.kt

6.9

修改了 FeedViewModel

加入了sqlite来存储点赞状态 还是有问题

重新设计了内外点赞同步的逻辑用ai辅助修改代码

主页 FeedScreen

```
observeAsState(viewModel.uiState) ← 自动重组
```

```
onLikeClick = { viewModel.toggleLike(item.id) }
```

| 同一个 FeedViewModel 单例

详情 DetailScreen

```
var liked = remember { item.isLiked() } ← 本地状态
```

```
onClick → liked = !liked + viewModel.toggleLike()
```

```
FeedViewModel.toggleLike(feedId)
```

```
repository.toggleLike()
```

→ SQLite 持久化

→ FeedItem.setLiked() ← 同一引用

```
uiState.postValue()
```

→ FeedScreen 重组，图标更新

```
likeSyncEvent.postValue()
```

→ 详情页可监听同步

6.10

主页搜索框功能开发

- 在 FeedScreen 顶部（分类 Tab 与信息流列表之间）新增搜索框组件，采用 Jetpack Compose + Material 3 实现
- 支持文字输入、一键清除、搜索图标引导等交互细节
- 搜索关键词状态通过 `remember + mutableStateOf` 在 Composable 层管理，与 ViewModel 解耦
- `onSearch` 回调已预埋，后续可对接 FeedRepository → AiApiService 搜索接口
- 修复了 FeedScreen 原有中文乱码问题（标题、分类标签等）

6.11

创建 `AiConfig.java` 独立配置文件，`MainActivity` 转为 Java，API Key / URL / Model 集中管理

AI 搜索功能：新增 `AiSearchService`，调用 AI 提取搜索关键词 → 本地模糊匹配标题/描述/标签；`FeedScreen` 搜索栏、键盘搜索键、清除按钮全部接通

线程架构：将原先共用的单线程池拆为三个独立 Executor：

- `refreshExecutor` — 下拉刷新，最高响应优先级，不被 AI 生成阻塞
- `searchExecutor` — 搜索独立线程，不受数据加载排队影响
- `dataExecutor` — 数据加载 + AI 摘要异步生成

刷新体验优化：每次刷新重新生成并随机打乱 mock 数据

存在问题：刷新/搜索存在卡顿，要优化线程的流程

6.12

重新设计了状态同步逻辑，改为三线并行，这样就可以减少卡顿，重新完成刷新模块

Main Thread (Compose UI)

↑ `StateFlow.collectAsState()` 自动观察

|

viewModelScope (协程作用域，跟随 ViewModel 生命周期)

|—— `refresh()` → `viewModelScope.launch` → `Dispatchers.Main` (默认)

| |—— `loadFeedItems()` → 同步生成 Mock 数据

| |—— `generateInsightsFor()` → 串行调 AI API (在协程内挂起，不阻塞 UI)

- | └── delay(remaining) → 保证刷新动画
- |
- |─── loadMore() → 同上，追加分页
- |
- |─── search() → viewModelScope.launch
 - | |─── AiSearchService.extractKeywords() → 请求云端拆词（协程阻塞等待）
 - | └── 本地 filter 匹配
- |
- |─── generateInsightsFor() → 独立的 aiInsightJob
 - | |─── for 循环遍历 items → 逐个调 AiInsightGenerator.generate()
 - | |─── 每个生成完 → allItems.value = allItems.value.map { ...copy(insight=...) }
 - | └── refresh() 时 cancel() 旧 job，启动新 job

6.13

详细的看了别人的例子和ai说的综合了一下，修改点赞同步以及存储逻辑

FeedViewModel (全局单例)

```

|
allItems: StateFlow<List<FeedItem>>

```

```

┌──────────────────┐
└──┴──┴──┴──┴──┴──┘
  ↓    ↓

```

FeedScreen.kt DetailScreen.kt

collectAsState() collectAsState()

```

  ↑      ↑

```

点 like 按钮 点 like 按钮

```

┌─── toggleLike(id) ───┐
|

```

```

allItems.value = newList

```

```

|
┌──────────────────┐
└──┴──┴──┴──┴──┴──┘
  ↓    ↓

```

FeedScreen 重绘 DetailScreen 重绘

SQLite 写入 → 仓库内存更新 → 返回不可变新对象

新列表赋值给 `allItems` → **所有观察者** (FeedScreen + DetailScreen) 自动收到更新

6.14

完成了视频播放，视频播放最后选用的通过url访问视频，加载会有点慢

设计了播放复用

FeedScreen.kt

- |—— rememberAutoPlayVideoId() → 计算 $\geq 60\%$ 可见的 VIDEO 卡片 id
- |—— rememberPreloadVideoIds() → 计算所有可见 VIDEO 卡片 id 集合
- |
- |—— AdCardFactory(item, shouldAutoPlayVideo, shouldPreloadVideo)



VideoAdCard

- |—— FeedVideoPlayerPool.acquire(context, itemId) → ExoPlayer 复用
- | |—— mutableMapOf<String, ExoPlayer>
- |
- |—— VideoCacheManager.getPlayableVideoUri(url) → 缓存命中/在线
- | |—— 命中: file:///cacheDir/video_cache/{sha256}.mp4
- | |—— 未命中: <https://videos.pexels.com/...>
- |
- |—— player.setMediaItem(MediaItem.fromUri(uri))
- |—— player.prepare() → player.playWhenReady = true
- |
- |—— UI 控制
- | |—— PlayArrow 呼吸动画按钮
- | |—— VolumeOff/VolumeUp 静音切换
- | |—— Fullscreen → Dialog(全屏 PlayerView + 控制器)

待解决的的问题

- ☒ ~~下拉刷新还是固定的，需要改成随机序号~~
- ☒ ~~外页面点赞有问题，内外同步有问题~~
- ☒ ~~刷新反应慢，会等一会~~
- ☐ 视频反应慢
- ☒ ~~搜索后没有返回原位置~~
- ☒ 刷新没有变化
- ☒ 刷新动画
- ☒ ~~加了几张图片后，全部显示为小图子~~

需要开发的功能

- ☒ ~~api接入生成标签、总结概要~~
- ☒ 点赞数量统计
- ☒ 搜索功能
- ☒ 分享/收藏
- ☒ ~~api实现模糊搜索~~
- ☒ 视频